

Project Executable Files

Date	04 July 2026
Team ID	XXXXXX
Project Name	Credit Card Approval Prediction System
Maximum Marks	3 Marks

Project Executable Files

This section captures submission of all executable and deployable files for the Credit Card Approval Prediction System project. All files are organized, named, and uploaded to the specified submission platform so the evaluator can run or deploy the solution using the provided files and instructions.

Step 1: Submission Checklist

S.No	Item to Submit	Submitted (Yes / No / NA)
1	Complete source code (all files and folders)	Yes
2	README / Setup Guide (instructions to run the project)	Yes
3	requirements.txt / package.json / dependency file	Yes
4	Database schema / seed files (if applicable)	NA
5	Environment configuration file (.env.example or similar)	Yes
6	Deployed application URL (if hosted)	NA
7	APK / Executable binary (if applicable for mobile/desktop apps)	NA
8	Dockerfile / Containerization config (if applicable)	NA
9	Test files and test results	Yes
10	Demo video or walkthrough (if required)	Yes

Step 2: File / Folder Structure

Overview of the project's file and folder structure:

app.py Main Flask application that handles routing, prediction, authentication, and session management. train_model.py Generates the dataset, preprocesses the data, trains machine learning models, and saves the best-performing XGBoost model. requirements.txt Lists all Python libraries required to run the project. dataset/ Stores the generated credit card approval dataset used for training the machine learning model. model/ Contains the trained machine learning model, scaler, and encoder used during prediction. templates/ Stores all HTML pages rendered by the Flask application. static/ Contains CSS, JavaScript, images, and other static assets used by the frontend. users.db SQLite database used to store registered user accounts securely. README.md Provides setup instructions, project description, and usage guidelines.

Step 3: Deployment / Access Details

Field	Details
Hosted / Deployed URL	Local: http://127.0.0.1:5000/ LAN: http://10.160.241.252:5000/
Login Credentials (Demo)	https://drive.google.com/file/d/1uUT1WmZGR0G69UCkW-ekb_zLNIxS4JfT/view?usp=sharing
Platform / Hosting Provider	IBM Watson Machine Learning (model hosting) with Flask app deployed on Render
Repository Link	https://github.com/TejeshSwamyReddy/Credit-Caed-Approval-Prediction
Demo Video Link	https://drive.google.com/file/d/1Vq--_6I7oIhyLB1Jt0V84k37w2-k60IO/view?usp=sharing

Step 4: Run Instructions

Step-by-step instructions for the evaluator to run the project locally.

Step 5: Known Issues / Limitations

1. Install Python 3.10 or above on your system and ensure Python is added to the system PATH.
2. Download or clone the Credit Card Approval Prediction project and open it in Visual Studio Code or any Python IDE.
3. Open the project folder in the terminal and create a virtual environment if required.
4. Install all the required dependencies using the `requirements.txt` file.
5. Run the `train_model.py` file to generate the dataset and train the XGBoost model.
6. Ensure that the trained model, scaler, and encoder files are successfully created in the model folder.
7. Start the Flask application by executing the `app.py` file from the terminal.
8. Open a web browser and navigate to <http://127.0.0.1:5000/> to access the application.
9. Register a new user account or log in using existing credentials, enter the applicant details, and click **Evaluate Application Risk** to generate the prediction.
10. View the prediction result along with the confidence score and explanation, check the Prediction History if required, and close the application after testing is completed.

Step 5: Known Issues / Limitations

S.No	Known Issue / Limitation	Workaround / Status
1	Prediction accuracy drops slightly for applicants with very limited credit history (thin-file applicants).	Flagged for manual review by the credit analyst; documented as a future improvement area.
2	IBM Watson ML endpoint occasionally has cold-start latency on the first request after inactivity.	Added a loading indicator on the web app; subsequent requests respond in real time.
3	The app currently supports English-language input only.	Out of scope for this version; noted as a potential future enhancement.

